TESI DI LAUREA MAGISTRALE IN INFORMATICA

Il Problema del Neighborhood Search su Grafi Temporali

Franco Salvucci - *0368692*

Relatore: Prof. Luciano Gualà **Correlatore**: Dott. Maestro Alessandro Straziota

Università degli Studi di Roma "Tor Vergata"

A.A. 2025/2026

Percorso della presentazione

01
Contesto
Definizioni e preliminari

02
Soluzioni proposte
Analisi e bound ottenuti

03
Esperimenti
Risultati e confronti

04
Conclusioni
Contributi e sviluppi futuri

SEZIONE 01

Fondamenti e definizione dei problemi

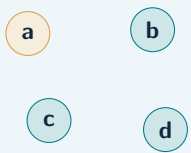
Neighborhood Search statico, MinHash/LSH e passaggio alla dimensione temporale

Neighborhood Search sui grafi statici

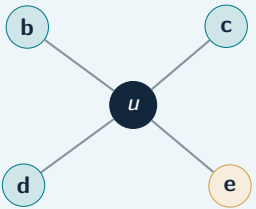
Definizione del problema: Dati $G = (V, E)$ un grafo, $S \subseteq V$ un sottoinsieme di nodi, $\tau \in [0, 1]$ un valore soglia e $\sigma : 2^V \times 2^V \to [0, 1]$ una funzione di similarità, trovare un nodo $u \in V$ tale che

$$\sigma(S, N(u)) \geq \tau$$

Insieme della query S



Vicinato del candidato u

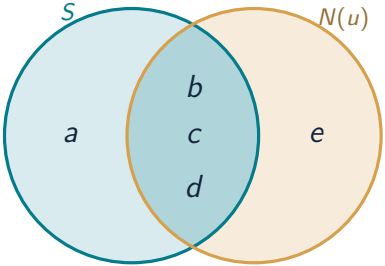


σ indica una misura di similarità generica.

La similarità scelta: indice di Jaccard

Dalla formulazione generica $\sigma(S, N(u))$ alla misura insiemistica

$$\sigma(S, N(u)) = J(S, N(u)) = \frac{|S \cap N(u)|}{|S \cup N(u)|}$$



Esempio

$$J(S, N(u)) = \frac{3}{5} = 0.6$$

Tre elementi in comune su cinque distinti.

MinHash

$\Pr[h_{\min}(S) = h_{\min}(N(u))] = J(S, N(u))$

k funzioni $\rightarrow$ signature compatta.

Neighborhood Search mediante MinHash e LSH

1. Sketch MinHash di S e di ogni $N(u)$

↓

2. Suddivisione in b bande di r righe

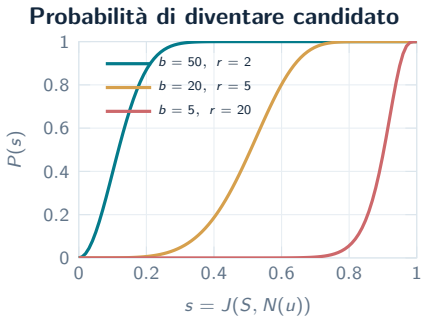
↓

3. Bucket LSH: almeno una banda uguale

↓

4. Verifica finale sui soli candidati

La verifica può calcolare Jaccard esatta o stimarla dalla firma completa.

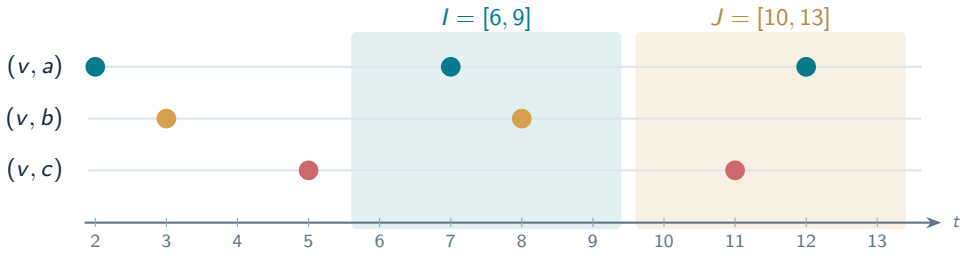


$k = br$; $P(s) = 1 - (1 - s^r)^b$.

È facile notare che la soluzione proposta funziona sia con S sia con un singolo nodo v. Per semplicità di trattazione, da qui in avanti useremo $S = v$

Grafi temporali: attività e vicinati

In un **grafo temporale** $\mathcal{G} = (V, E, \lambda)$, ogni arco $e \in E$ è presente negli istanti $\lambda(e)$. Gli **archi temporali** sono le triple $(u, w, t) \in \mathcal{E}$, con $t \in \lambda((u, w))$.



Istanti di attività di v

$\Lambda(v) = \{t : \exists u, (v, u, t) \in \mathcal{E}\}$

Nell'esempio: $\{2, 3, 5, 7, 8, 11, 12\}$.

Vicinato nell'intervallo I

$N^I(v) = \{u : \exists t \in I, (v, u, t) \in \mathcal{E}\}$

Nell'esempio: $N^I(v) = \{a, b\}$, $N^J(v) = \{a, c\}$.

Due varianti dello stesso problema

Dato $\mathcal{G} = (V, E, \lambda), v \in V, \tau \in [0,1]$ **e** $\sigma : 2^V \times 2^V \to [0,1]$ **trovare** $u \neq v$ **ed un intervallo** $I = [t, t+\mu]$ **tali che** $J(N^I(v), N^I(u)) \geq \tau$

VARIANTE 1

Ampiezza fissa

μ viene passato come **input del problema**

Indice progettato per quella finestra

VARIANTE 2

Ampiezza variabile

μ fornito con **la query**

Serve supportare più ampiezze

SEZIONE 02

Soluzioni proposte

Pruning, indicizzazione LSH modificata e oracolo $\mathcal{O}$

Variante 1: due ingredienti fondamentali

01 INDICE LSH UNIFICATO

Pruning temporale $W(v)$

Un unico insieme di tabelle con chiave

$\big(\mathrm{Hash}_j(B_j(v,t)), t\big)$

Riduce finestre e confronti

02 ORACOLO $\mathcal{O}$

Recupero degli sketch temporali

Range Tree sugli istanti attivi

$\mathrm{Sketch}(N^{[t,t+\mu]}(v))$

Combina sketch precomputati



Variante 1: risultati teorici della soluzione

μ è fissata in preprocessing; $w_v = |W(v)|$, $m_v = |\Lambda(v)|$,
$C_v = \sum_{t \in W(v)} |C(v,t)|$ è il totale dei candidati esaminati.

Building time

$$O\left(\sum_{v \in V} w_v(k \log m_v + b)\right)$$

Costruzione indice LSH

Spazio

$$O\left(b \sum_{v \in V} w_v\right)$$

Memoria indice LSH

Query time

$$O(w_v(k \log m_v + b) + kC_v)$$

Per una query su v

$$w_v \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}$$

Bound dell'indice LSH con pruning; la costruzione e lo spazio dell'oracolo $\mathcal{O}$ sono contabilizzati separatamente.

Variante 1: il pruning temporale $W(v)$

$$W(v) = \{t : \ [t, t+\mu] \cap \Lambda(v) \neq \varnothing\}$$

Esempio $\mu = 2$

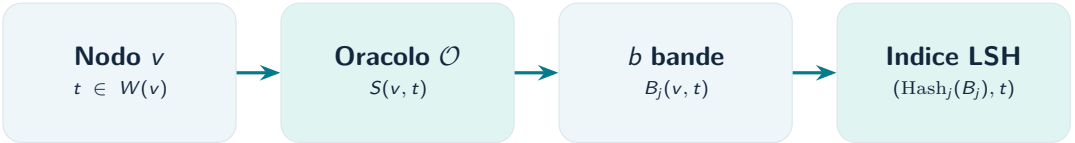


Finestre senza eventi **escluse**

Finestre con eventi **mantenute**

Ogni istante attivo genera al più $\mu + 1$ partenze rilevanti: $|W(v)| \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}$.

Variante 1: l'indice LSH unificato



Indice globale

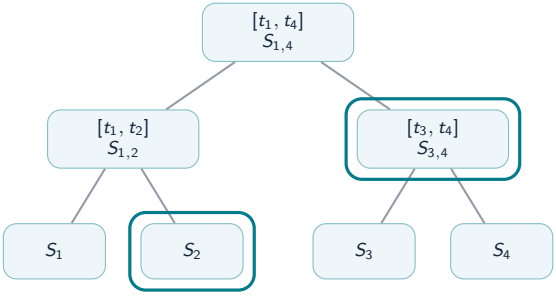
Tutte le coppie (v, t) rilevanti condividono le stesse b tabelle.

Timestamp nella chiave

Si confrontano candidati della **stessa finestra**.

Filtro LSH $\longrightarrow$ confronto delle signature complete

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Query $[t_2, t_4]$: merge di S_2 e $S_{3,4}$

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

componibili via minimi

$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Variante 2: ampiezza nota soltanto alla query

Difficoltà

$\Theta(T^2)$ intervalli complessivi

Un indice LSH per ogni μ sarebbe costoso.

Risultato proposto

Filtro euristico sulla **prima componente MinHash**.

Corrispondenze MinHash$_1$ sulle finestre

↓

Rettangoli geometrici associati ai nodi

↓

Range Tree multidimensionali $\rightarrow$ candidati

↓

Verifica successiva con firma completa

Variante 2: bound dell'oracolo sul primo MinHash

$N \leq nT$: rettangoli indicizzati; $k_v(\mu) \leq T$: rettangoli rilevanti per la query;
$M = |C(v, \mu)|$: coppie nodo–finestra riportate dal filtro.

Building time

$$O\big(|\mathcal{E}| + nT + N \log^3 N\big)$$

Costruzione rettangoli + Range Tree 4D

Spazio

$$O\big(nT + N \log^3 N\big)$$

Indice geometrico

Query del filtro

$$O\big(k_v(\mu) \log^3 N + M\big)$$

Prima componente MinHash

$$O\big(T \log^3(nT) + M\big)$$

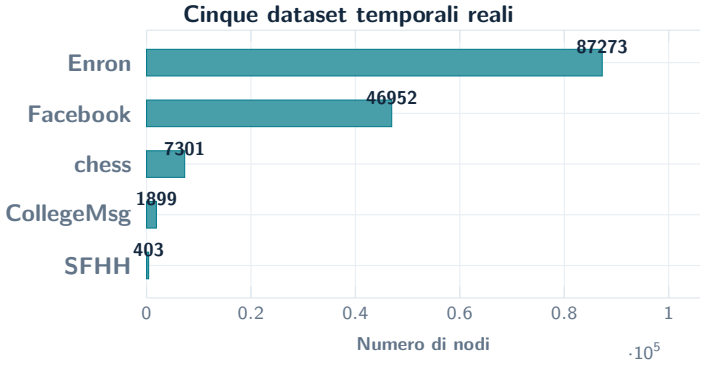
Bound semplificato della ricerca dei candidati. Non comprende la verifica finale con le signature complete, né lo spazio dell'oracolo $\mathcal{O}$ usato per recuperarle.

SEZIONE 03

Valutazione sperimentale

L'oracolo, il pruning e i confronti con la Brute Force

Configurazione degli esperimenti



Parametri principali

$\tau = 0.35$
$k = 128$
$b = 8,\ r = 16$

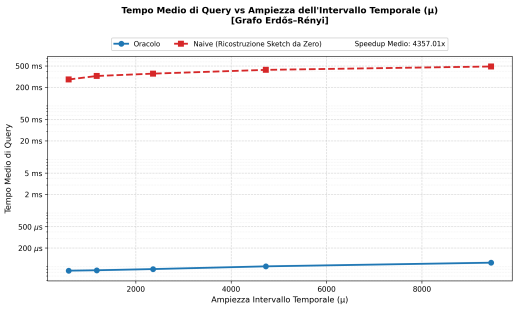
Strategie a confronto

BF
LSH+W
LSH–W
Alt+W

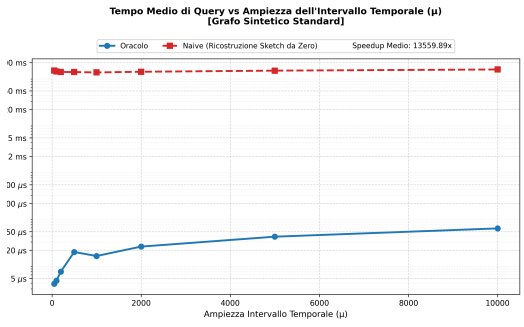
Fino a 8000 sorgenti distinte; le quattro strategie condividono l'oracolo $\mathcal{O}$.

L'oracolo $\mathcal{O}$ evita la ricostruzione del vicinato

Erdős–Rényi (10 000 nodi)



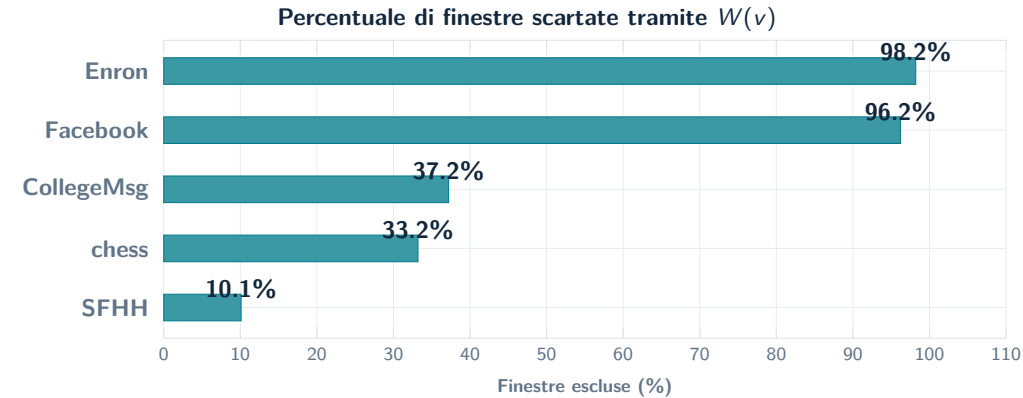
Grafo sintetico (300 000 nodi)



Sketch tramite oracolo: microsecondi. Ricostruzione naïve: millisecondi.

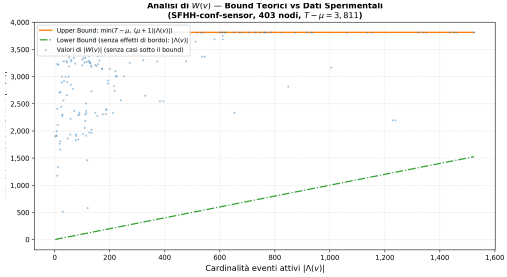
Figure estratte dalla tesi, Fig. 7; il confronto riguarda il recupero degli sketch.

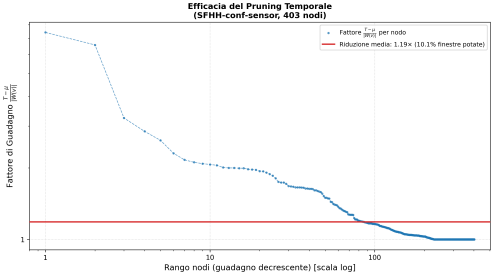
Pruning: quante finestre vengono eliminate?



Fonte: Tabella 2 e Fig. 8 della tesi. Il pruning non garantisce lo stesso guadagno su ogni distribuzione temporale.

Pruning moderato: SFHH-conf-sensor

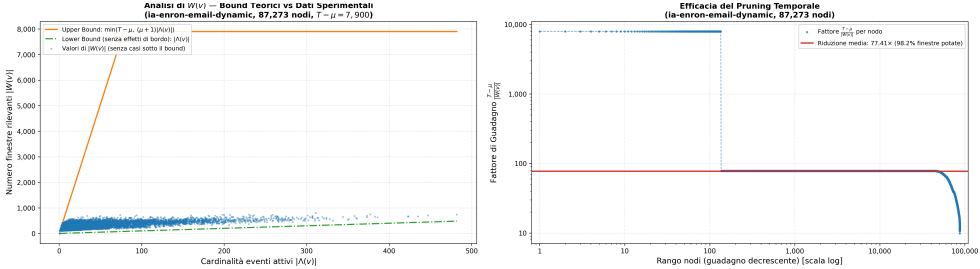




Solo il 10.1% delle finestre eliminate **Guadagno medio** $1.19\times$

Figura 8(b) della tesi. Con eventi distribuiti densamente, $W(v)$ rimane vicino all'insieme delle finestre ammissibili.

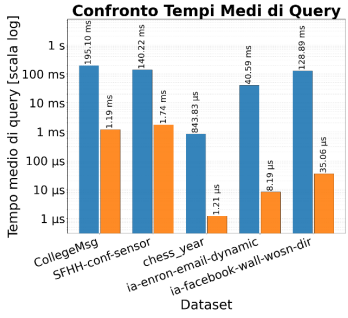
Pruning molto efficace: ia-enron-email-dynamic

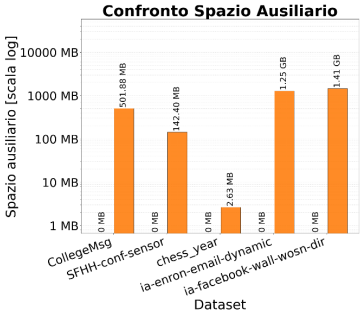


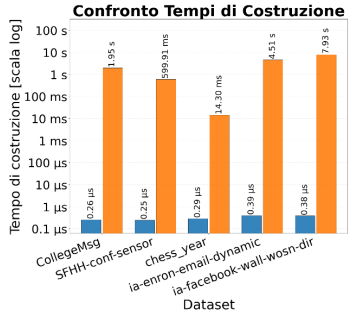
98.2% delle finestre eliminate **Guadagno medio** $77.41\times$

Figura 8(d) della tesi. Con attività temporalmente sparsa, la ricerca scarta la grande maggioranza delle finestre.

Benchmark: Brute Force e soluzione proposta







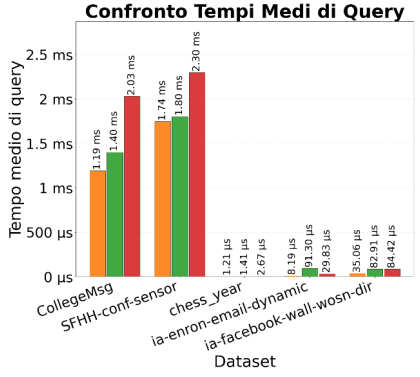
Baseline (Brute Force) | Mia Soluzione (LSH Unificato)

Enron: circa 4956× più veloce

Facebook: circa 3676× più veloce

Confronto sui cinque dataset: tempi medi di query, spazio ausiliario e tempi di costruzione. Le scale logaritmiche sono indicate nei grafici.

Confronto fra le tre strategie LSH: query



LSH+W

Indice globale con pruning temporale.

LSH–W

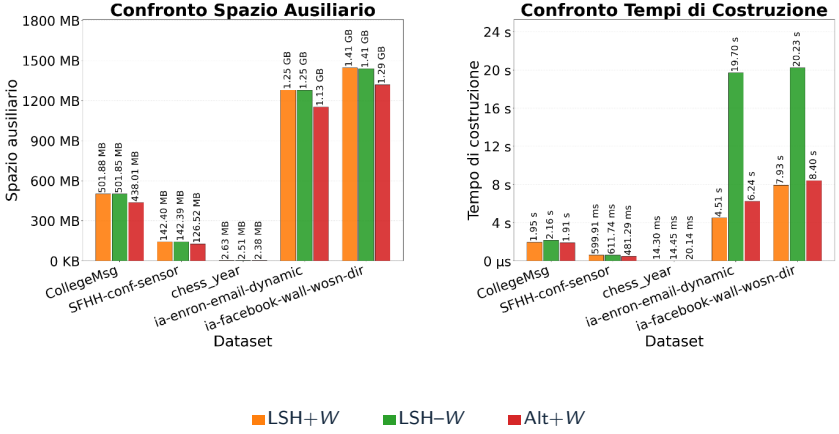
Lo stesso indice, senza pruning.

Alt+W

Indici LSH distinti per finestra.

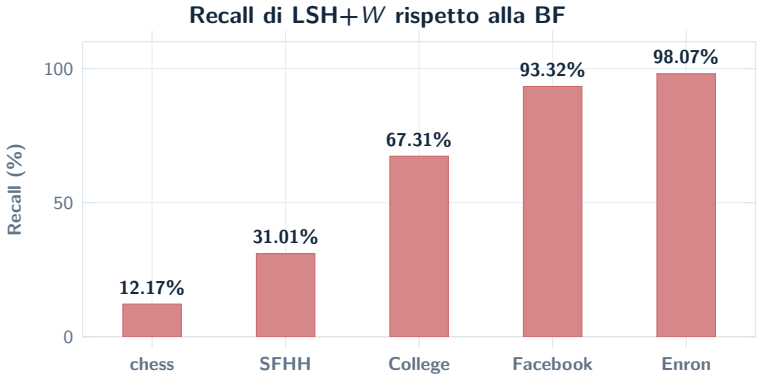
Il pruning è particolarmente efficace su Enron: il tempo medio scende da 91.30 a 8.19 μs.

Il costo del preprocessing e della memoria



Spazio ausiliario e tempo di costruzione sui cinque dataset, da valutare insieme ai tempi di query.

La velocità introduce un compromesso: la recall



Confronto relativo alle stesse signature MinHash (non al Jaccard esatto). Tab. 5 della tesi.

0

falsi positivi vs BF

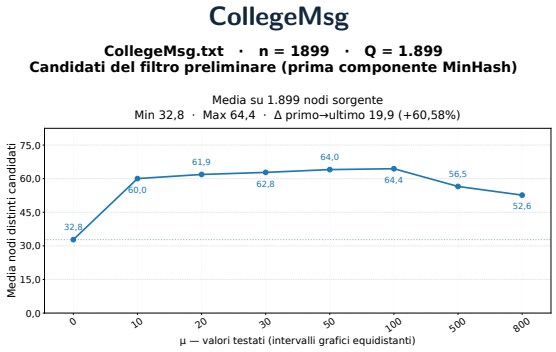
Falsi negativi

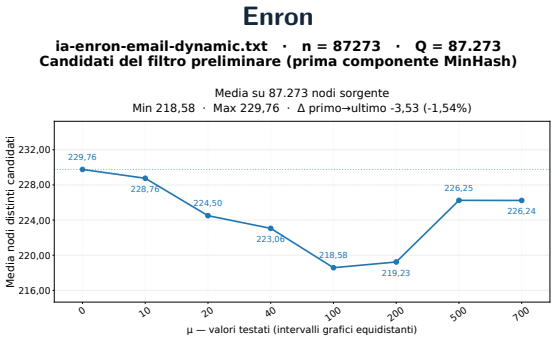
Alcuni nodi simili non diventano candidati LSH.

Recall

$$\text{Recall}_{\text{LSH}|\text{BF}} = \frac{|B \cap L|}{|B|}, \qquad |B| > 0.$$

Variante 2: il filtro geometrico riduce i candidati





Su Enron circa 220–230 candidati distinti in media.

Fig. 11: è misurato soltanto il filtro preliminare $|C_1(v, \mu)|$, non la verifica completa, la recall o la velocità dell'intera query.

SEZIONE 04

Conclusioni e sviluppi futuri

Contributi, limiti sperimentali e direzioni di ricerca

Risultati principali e prossimi sviluppi

Pruning temporale

98.2%

finestre escluse su Enron

Velocità della query

8431×

speedup massimo rilevato

Compromesso recall

12–98%

recall dipendente dal dataset

Sviluppi futuri

Esplorare altri framework *Filter&Verify* • Valutare la verifica completa nella variante 2 • Supportare aggiornamenti dinamici



Grazie per l'attenzione!

Franco Salvucci